DIDATTICA INTEGRATIVA

Ottimizzazione: Modelli e Metodi

Laurea triennale in Statistica, secondo anno
Università degli Studi di Padova

Introduzione ai laboratori

Anthony Palmieri
Dottorando in Computational Mathematics

Quali attività finanziare con 1.000 euro?

	Attività	Costo (€)	Beneficio
A	Torneo sportivo	400	8
B	Laboratorio foto	300	6
C	Rassegna cinema	500	9
D	Scambio libri	200	5
E	Concerto	600	11

Dati noti

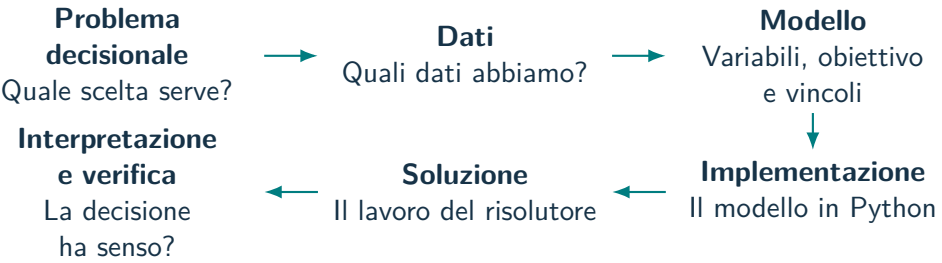
Costi, benefici e budget.

Decisione

Quali attività realizzare, ciascuna per intero.

Obiettivo: ottenere il beneficio totale più alto, rispettando il budget.

Il percorso di un problema di ottimizzazione



Nel corso: fondamenti e metodi dell'ottimizzazione.

Nei laboratori: piccoli modelli lineari interi e binari, fino alla verifica del risultato.

Che cosa impareremo a fare

1. Eseguire un notebook Colab in modo ordinato e riproducibile.
2. Organizzare piccoli insiemi di dati con Python.
3. Tradurre un testo in variabili, obiettivo e vincoli.
4. Implementare piccoli modelli interi o binari con `gurobipy`.
5. Leggere lo stato della risoluzione e verificare una soluzione disponibile.

Un ripasso Python mirato agli esercizi

Strumento	Che cosa useremo	Perché ci serve
Python	Liste, dizionari, indici, cicli, condizioni, semplici funzioni e comprensioni	Descrivere dati e operazioni
NumPy	Array, dimensioni, operazioni numeriche essenziali	Lavorare con vettori e matrici
pandas	Piccoli CSV, selezione e organizzazione di tabelle	Preparare i dati di un modello
SciPy	Breve orientamento al calcolo scientifico e a `scipy.optimize`	Collocare gli strumenti disponibili

Useremo gli strumenti utili al singolo esercizio. Queste librerie non sono tutte necessarie per usare `gurobipy`.

Python, Colab e Gurobi hanno ruoli diversi

Python	Il linguaggio con cui scriviamo il programma.
Colab	L'ambiente online in cui lavoriamo con i notebook.
`gurobipy`	L'API Python: l'interfaccia per costruire i modelli e usare Gurobi.
Gurobi Optimizer	Il risolutore che cerca soluzioni e ne valuta l'ottimalità.

Gurobi è un software commerciale con opzioni gratuite.
Il nostro focus: ottimizzazione lineare intera e mista intera.

Fonte: Gurobi, Python API.

Il primo notebook in Colab

1. Aprire Colab e accedere con un account Google.
2. Creare un notebook o aprire quello fornito.
3. Dal menu **File**, salvarne una copia in Drive.
4. Eseguire una cella con il pulsante di avvio o **Shift+Invio**.

Fonte: Colab, FAQ.

Celle di testo

Spiegazioni, formule e commenti al risultato.

Celle di codice

Istruzioni Python eseguibili.

```
budget = 1000
print(budget)
```

Il notebook salvato e il runtime

Notebook salvato

Conserva testo, codice e gli eventuali output salvati.

Si può riaprire e condividere.

Runtime: la sessione di calcolo

Contiene variabili in memoria, file temporanei e pacchetti installati.

Quando viene ricreato, questo stato va ricostruito.

Prova di riproducibilità: runtime pulito, installazioni e dati, poi esecuzione di tutte le celle dall'inizio.

Useremo una CPU standard. Non servono GPU, TPU o abbonamenti.

Fonte: Colab, FAQ e limiti delle risorse.

La selezione delle attività diventa un modello

$J = \{A, B, C, D, E\}$: insieme delle attività.
c_j: costo in euro; b_j: beneficio in punti; $K = 1000$: budget.

$$x_j = \begin{cases} 1 & \text{se realizziamo l'attività } j, \\ 0 & \text{altrimenti.} \end{cases}$$

$$\begin{aligned} \max_x \quad & \sum_{j \in J} b_j x_j && \text{beneficio totale} \\ \text{s.t.} \quad & \sum_{j \in J} c_j x_j \leq K && \text{budget} \\ & x_j \in \{0, 1\} \quad (j \in J) && \text{scelte indivisibili} \end{aligned}$$

Che significato avrebbe scegliere metà progetto?

Una soluzione da interpretare e verificare

Scelta ottima

B, C e D

Laboratorio foto, rassegna cinema e scambio libri.

$$x^* = (0, 1, 1, 1, 0)$$

Ordine delle componenti: A, B, C, D, E.

Controlli sul risultato

Costo

$300 + 500 + 200 = 1000$ euro.

Beneficio

$6 + 9 + 5 = 20$ punti.

Decisioni

Tutte le componenti sono binarie.

A, B e D costano 900 euro e valgono 19 punti: una scelta ammissibile può essere migliorabile.

Il ciclo essenziale con gurobipy

	Operazione	Oggetto o comando Python
1	Creare il modello	`m = gp.Model("attivita")`
2	Aggiungere le variabili	`m.addVars(...)`
3	Definire l'obiettivo	`m.setObjective(...)`
4	Aggiungere i vincoli	`m.addConstr(...)`
5	Avviare la risoluzione	`m.optimize()`
6	Controllare e leggere	`m.Status`, `m.SolCount`, poi `X`

Fonte: Gurobi, Python API.

Soluzione disponibile e ottimalità

Stato del modello	Come interpretarlo
`GRB.OPTIMAL`	Soluzione ottima disponibile, entro le tolleranze del risolutore.
`GRB.INFEASIBLE`	Il modello è stato riconosciuto inammissibile. Controllare dati e vincoli.
`GRB.TIME_LIMIT`	Tempo esaurito. Può esserci una soluzione, senza una certificazione di ottimalità.

Prima di leggere `x[j].X` o `m.ObjVal`: `m.SolCount > 0`.

Nessuna soluzione trovata non significa necessariamente modello inammissibile.

Fonte: Gurobi, codici di stato.

Come cambia la decisione se cambia il modello?

Scenario	Attività ottime	Beneficio
Budget di 1.000 euro	B, C, D	20
Solo il budget sale a 1.200 euro	A, D, E	24
Budget di 1.000 euro e B incompatibile con C	A, E oppure A, B, D	19

L'incompatibilità si traduce nel vincolo $x_B + x_C \leq 1$.

Il risolutore risponde al modello che gli diamo.
Chi costruisce il modello deve scegliere dati, ipotesi e vincoli.

La configurazione proposta per i laboratori

Licenza limitata inclusa con `gurobipy`

Piccoli modelli didattici, senza chiavi personali.

```
!pip install gurobipy
import gurobipy as gp
from gurobipy import GRB
```

Licenza inclusa tramite pip. Gratuita e soggetta a condizioni d'uso. Per modelli lineari: al massimo **2.000 variabili e 2.000 vincoli**. Per ricerca, sviluppo e valutazione, non per applicazioni commerciali.

Academic WLS. Gratuita per accademici idonei e uso accademico non commerciale, compatibile con Colab e senza questi limiti dimensionali. Procedura opzionale in appendice A3.

Fonte: Gurobi su Colab.

Prima del primo laboratorio

- Computer, browser aggiornato e connessione.
- Accesso a Colab con il proprio account.
- Un notebook aperto e salvato come propria copia.
- Una semplice cella eseguita senza errori.

Una prova sufficiente

```
print(2 + 3)
```

Risultato atteso: 5.

Materiali

Notebook e materiali

Nei laboratori costruiremo il modello e controlleremo il significato della soluzione.

Riferimenti e approfondimenti

- Scheda ufficiale del corso, Università di Padova
- Google Colab: FAQ, notebook e runtime
- Gurobi su Colab: installazione e licenze
- Gurobi: licenze accademiche gratuite
- Gurobi: documentazione dell'API Python
- Stati della risoluzione Attributi del modello e SolCount
- Richiesta Academic WLS Requisiti e condizioni Academic WLS

A1. Esempio completo: dati e modello

Dopo l'installazione della slide 15, eseguire A1 e A2 in sequenza.

```
import gurobipy as gp
from gurobipy import GRB
J = ["A", "B", "C", "D", "E"]
c = dict(zip(J, [400, 300, 500, 200, 600]))
b = dict(zip(J, [8, 6, 9, 5, 11]))
budget = 1000
m = gp.Model("attivita")
x = m.addVars(J, vtype=GRB.BINARY, name="x")
m.setObjective(gp.quicksum(b[j] * x[j] for j in J),
               GRB.MAXIMIZE)
m.addConstr(gp.quicksum(c[j] * x[j] for j in J)
            <= budget, name="budget")
m.optimize()
print("Stato:", m.Status)
```

A2. Esempio completo: lettura e verifica

```
if m.SolCount > 0:
    scelti = [j for j in J if x[j].X > 0.5]
    costo = sum(c[j] for j in scelti)
    beneficio = sum(b[j] for j in scelti)
    print(scelti, "Costo:", costo, "Beneficio:", beneficio)
    assert costo <= budget
    assert abs(beneficio - m.ObjVal) <= 1e-6
    if m.Status == GRB.OPTIMAL:
        print("Ottimo certificato entro le tolleranze.")
    else:
        print("Soluzione disponibile: controllare lo stato.")
else:
    print("Nessuna soluzione disponibile: controllare lo stato.")
m.dispose()
```

Dati base: `['B', 'C', 'D']`, costo 1000, beneficio 20.

A3. Academic WLS: configurazione opzionale

1. Creare un account Gurobi accademico, se idonei.
2. Dalla rete universitaria o da una VPN completa, richiedere **WLS Academic** nel portale Gurobi.
3. In Web License Manager, generare e custodire la propria chiave.
4. Configurare un ambiente `gp.Env` con credenziali private. Creare il modello con `gp.Model(env=env)`.

Internet richiesto durante l'uso. Validità di 90 giorni, estendibile.
In Colab usare una versione corrente di `gurobipy`.

Chiave personale: non inserirla nei notebook condivisi o nei repository. Ogni studente usa la propria licenza.